Practical :08 **Date:09-10-2026**

Hyperparameter Optimization – I

Perform hyperparameter optimizations by using Grid and Random search techniques for Regression and classification problem.

Procedure:

Step 1: Open Visual Studio Code and create a new Python file.

Step 2: Import the required Python libraries.

Step 3: Load the dataset into Python.

Step 4: Separate the independent variables and dependent variable.

Step 5: Select a suitable machine learning model for the regression or classification problem.

Step 6: Define the hyperparameters and their possible values.

Step 7: Apply Random Search using RandomizedSearchCV() to find the best hyperparameters.

Step 8: Apply Grid Search using GridSearchCV() to find the best hyperparameters.

Step 9: Evaluate the models using cross-validation and compare their performance scores.

Step 10: Display the best hyperparameters and scores obtained from both search techniques.

Step 11: Repeat the procedure for the remaining datasets.

Step 12: Compare the results and draw conclusions about the performance of Grid Search and Random Search.

Solution:

```
import pandas as pd
from scipy.stats import loguniform
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import Ridge, LogisticRegression
from sklearn.model_selection import (
    GridSearchCV,
    RandomizedSearchCV,
    RepeatedKFold,
    RepeatedStratifiedKFold
)
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
url = "https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/insurance.csv"
df_reg = pd.read_csv(url)
print("Regression Dataset")
print(df_reg.head())
X_reg = df_reg[["age"]].to_numpy()
y_reg = df_reg["charges"].to_numpy()
reg_model = Ridge()
```

```
reg_cv = RepeatedKFold(
    n_splits=5,
    n_repeats=2,
    random_state=1
)
reg_random_params = {
    "alpha": loguniform(1e-4, 100),
    "fit_intercept": [True, False]
}
reg_random_search = RandomizedSearchCV(
    estimator=reg_model,
    param_distributions=reg_random_params,
    n_iter=20,
    scoring="neg_mean_absolute_error",
    cv=reg_cv,
    random_state=1,
    n_jobs=-1
```

```
)
reg_random_search.fit(X_reg, y_reg)
print("\nREGRESSION - RANDOM SEARCH")
print("Best Parameters:", reg_random_search.best_params_)
print("Best Score:", reg_random_search.best_score_)
reg_grid_params = {
    "alpha": [0.01, 0.1, 1, 10, 100],
    "fit_intercept": [True, False]
}

reg_grid_search = GridSearchCV(
    estimator=reg_model,
    param_grid=reg_grid_params,
    scoring="neg_mean_absolute_error",
    cv=reg_cv,
    n_jobs=-1
)
```

```
reg_grid_search.fit(X_reg, y_reg)
print("\nREGRESSION - GRID SEARCH")
print("Best Parameters:", reg_grid_search.best_params_)
print("Best Score:", reg_grid_search.best_score_)
data = load_breast_cancer()
X_cls = data.data
y_cls = data.target
cls_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=10000))
])
cls_cv = RepeatedStratifiedKFold(
    n_splits=5,
    n_repeats=2,
    random_state=1
)
```

```python
cls_random_params = {
    "classifier__C": loguniform(1e-4, 100),
    "classifier__solver": ["liblinear", "lbfgs"],
    "classifier__penalty": ["l2"]
}
cls_random_search = RandomizedSearchCV(
    estimator=cls_model,
    param_distributions=cls_random_params,
    n_iter=20,
    scoring="accuracy",
    cv=cls_cv,
    random_state=1,
    n_jobs=-1
)
```

```python
cls_random_search.fit(X_cls, y_cls)
print("\nCLASSIFICATION - RANDOM SEARCH")
print("Best Parameters:", cls_random_search.best_params_)
print("Best Accuracy:", cls_random_search.best_score_)
cls_grid_params = {
    "classifier__C": [0.01, 0.1, 1, 10, 100],
    "classifier__solver": ["liblinear", "lbfgs"],
    "classifier__penalty": ["l2"]
}
cls_grid_search = GridSearchCV(
    estimator=cls_model,
    param_grid=cls_grid_params,
    scoring="accuracy",
    cv=cls_cv,
    n_jobs=-1
)
```

```python
cls_grid_search.fit(X_cls, y_cls)
print("\nCLASSIFICATION - GRID SEARCH")
print("Best Parameters:", cls_grid_search.best_params_)
print("Best Accuracy:", cls_grid_search.best_score_)
print("\n===== FINAL COMPARISON =====")
print("\nRegression:")
print("Random Search Score:", reg_random_search.best_score_)
print("Grid Search Score:", reg_grid_search.best_score_)
print("\nClassification:")
print("Random Search Accuracy:"
print("Grid Search Accuracy:", cls_grid_search.best_score_)
```

(variable) cls_grid_search: GridSearchCV[Pipeline]

```
REGRESSION - RANDOM SEARCH
Best Parameters: {'alpha': np.float64(64.50144652189375), 'fit_intercept': False}
Best Score: -8922.996812367406

REGRESSION - GRID SEARCH
Best Parameters: {'alpha': 100, 'fit_intercept': False}
Best Score: -8922.885136919987

CLASSIFICATION - RANDOM SEARCH
Best Parameters: {'classifier__C': np.float64(0.17096232052870358), 'classifier__penalty': 'l2', 'classifier__solver': 'liblinear'}
Best Accuracy: 0.9798167986337525
```

```
CLASSIFICATION - GRID SEARCH
Best Parameters: {'classifier__C': 0.1, 'classifier__penalty': 'l2', 'classifier__solver': 'liblinear'}
Best Accuracy: 0.9806939916162086
```